

MAlgebra

Luc Alexander Lüthi

March 7th, 2026

Contents

1	Introduction	1
2	Substrate	1
2.1	Capabilities as Patterns	1
2.2	Control Capabilities	2
2.3	Signals and Metabolism	2
3	M(alware) Algebra	2
3.1	Primitives	2

1 Introduction

As a continuation of the note on Adversarial Systems, here I explore self metabolizing capability graph machines as a minimal exploitable substrate for studying adversarial systems, presenting a formalism for a malware algebra that allows the mathematical description and manipulation of systems through exploitable signals. These signals are metabolized by the structure of the system to perform computation, allowing emergent immunity (or security systems), and emergent parasitism (or malware). System structure is described by the same algebra as exploit signals, as signals themselves are simply small system mutations.

2 Substrate

Systems are simple directed graphs. Neither edges nor vertecies are significant or carry any meaning on their own. The significance of a graph comes from its structural invariants. The directed nature of the graph is significant, and two substructures with different directionalities are considered distinct structures. All structures are composed of nested substructures. A structure only contains what the flow of its directionality allows it to see.

2.1 Capabilities as Patterns

The capabilities associated with a system is defined by what structural capabiltiies it is able to see in its structural representation. Capabilities are each distinct substructures, which when present grant a system a certain capability. A core property of this formalism is that these capabilities can be both emergent and higher order. A higher order capability is present if a graph partially or fully normalizes down to an atomic structural capability. Normalization takes place when structures of the form $A \rightarrow B \rightarrow C$ reduce to structures of the form $A \rightarrow B$. Structures are allowed to manifest as capabilities under partial normalization rather than fully resolved normalization. A fully resolved normalization occurs when the structure cannot be reduced any further. Normalization occurs purely as a conceptual truth

about the structure of some system, and showing normalization does not mutate the structure. Some system definitions may include a capability which allows for higher order graph reasoning and conceptual normalization, others may choose to allow it to be a native property of the graph substrate without the need for an earned capability.

2.2 Control Capabilities

The presense of an observer, or the central identifying feature of an entity in these structural topologies, is simply another capability. A control structure is a capability which marks the presense of an actionable structural entity. These entities may send signals to other entities which are interpreted in a metabolic process. Control structures may be nested in the structure of a larger structural system owned by another entity. These entities may be cooperative and may send signals to eachother, or may be parasitic and act as viruses, spreading themselves and escalating their capabilities within a structural subsystem. Multiple instances of the same control structure may exist and represent different manifestations of the same controlling entity.

The strategy of an entity represented by a control structure is an externalized process from this substrate. This may be implemented as players playing a game, or a bot algorithm performing some heuristic based prerogative. Separate instances of equivalent control structures are accessible and actionable by the same "player", ie they represent the same entity.

2.3 Signals and Metabolism

Signals are represented by a pair of graph structures. The first is a structural pattern. This pattern will be queried for a match in the target graph, if the substructure exists, the matched location will be mutated to match the second graph in the signal. This process is called metabolism. Metabolism can occur on the first instance found of a depth first or breadth first traversal, which is also specified by the signal.

3 M(alware) Algebra

3.1 Primitives

There is one primitive type of thing in the M algebra, a graph view. A graph view consists of a graph, and a subsection of the graph considered selected. Graphs are defined by their adjacency matrices. Defining a graph by its adjacency matrix instantiates a view of the entire graph. Specific selection is done concatenatively, and symbols may be subscripted with the selection heuristic, either depth first or breadth first.

$$\begin{aligned}\gamma &= K_5 \\ \beta &= \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \\ \alpha &= \gamma\beta_b\end{aligned}$$

Beyond explicit selection there is one more primitive operation, ϕ replacement, which replaces the specified adjacency matrix in the selection of a view of its left operand with the selected graph of the view of its right operand.

$$\alpha\Phi \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

This affords us emergent graph based structure generation and computation.